Лабораторна робота № 4

Дослідження дерев рішень та ансамблевих методів
у задачі багатокласової класифікації

за наявності пропущених значень

Мета роботи

Дослідити та порівняти три алгоритми класифікації — дерево рішень (`Decision Tree`), випадковий ліс (`Random Forest`) та градієнтний бустинг (`XGBoost`) — на реальному датасеті `Annealing`, що містить пропущені значення. Основну увагу приділити:

- впливу способу обробки пропусків на якість класифікації;
- коректному (без витоку даних) підбору гіперпараметрів за допомогою Optuna;
- неупередженому оцінюванню моделей (вкладена крос-валідація) та статистичному порівнянню результатів;
- інтерпретації моделей і розбіжностей між їхніми прогнозами (permutation importance, SHAP).

Очікувані результати навчання

Після виконання роботи студент повинен уміти:

1. аналізувати структуру пропусків у даних і формулювати гіпотезу щодо механізму їх виникнення (MCAR / MAR / MNAR);
2. будувати пайплайни попередньої обробки в scikit-learn, що виключають витік даних;
3. налаштовувати гіперпараметри так, щоб оцінка якості моделі не була оптимістично зміщеною;
4. обґрунтовано порівнювати моделі з урахуванням дисперсії оцінок, а не лише середніх значень;
5. пояснювати як глобальну поведінку моделі, так і окремі прогнози.

Програмне забезпечення

Python $\geq$ 3.10, scikit-learn $\geq$ 1.4, xgboost $\geq$ 2.0, optuna $\geq$ 3.0, shap $\geq$ 0.44, pandas, matplotlib/seaborn. Версії бібліотек, використаних у роботі, необхідно зафіксувати у звіті (наприклад, `pip freeze > requirements.txt`).

Індивідуальний варіант

Усі джерела випадковості (поділ даних, крос-валідація, моделі, семплер Optuna) ініціалізуються однією константою:

$$\text{SEED} = 42 + N,$$

де N — номер студента у списку групи. Через це числові результати в кожного студента будуть різними, а висновки мають ґрунтуватися на власних результатах.

1 Короткі теоретичні відомості

1.1 Механізми виникнення пропусків

За класифікацією Рубіна [5] розрізняють три механізми:

- **MCAR** (Missing Completely At Random) — ймовірність пропуску не залежить ні від спостережуваних, ні від неспостережуваних даних;
- **MAR** (Missing At Random) — ймовірність пропуску залежить лише від спостережуваних ознак;
- **MNAR** (Missing Not At Random) — ймовірність пропуску залежить від самого відсутнього значення або від неспостережуваних чинників.

Якщо пропуски не є MCAR, сам факт пропуску може нести інформацію про цільову змінну. Саме тому бінарні індикатори пропусків (варіант B нижче) здатні поліпшити якість прогнозу [7]. В окремих датасетах частина пропусків є структурними: ознака просто не застосовна до цього об'єкта, і її «заповнення» медіаною чи модою не має фізичного змісту.

1.2 Оптимістичне зміщення під час підбору гіперпараметрів

Якщо гіперпараметри обрано за найкращим середнім значенням метрики на крос-валідації, то це саме значення є оптимістично зміщеною оцінкою якості: ми обрали максимум серед багатьох зашумлених оцінок [8]. Неупереджену оцінку дає вкладена крос-валідація (nested CV): зовнішній цикл оцінює якість, внутрішній — підбирає гіперпараметри лише на тренувальній частині кожного зовнішнього блоку.

1.3 Порівняння моделей за результатами крос-валідації

Блоки крос-валідації перетинаються з тренувальними даними, тому оцінки на різних блоках не є незалежними, і звичайний парний t-тест занижує дисперсію. Для повторної k-блочної крос-валідації (r повторень) використовують скоригований t-тест Наде–Бенжіо [9]:

$$t = \frac{\bar{d}}{\sqrt{\hat{\sigma}_d^2 \left(\dfrac{1}{kr} + \dfrac{n_{\text{test}}}{n_{\text{train}}} \right)}}, \qquad \text{ступені свободи: } kr - 1,$$

де d_j — різниця метрик двох моделей на j-му блоці (обидві моделі оцінюються на одних і тих самих блоках), $\bar{d}$ і $\hat{\sigma}_d^2$ — вибіркові середнє та дисперсія величин d_j, а $n_{\text{test}}/n_{\text{train}}$ — відношення розмірів валідаційної й тренувальної частин блоку.

2 Опис датасету

Датасет `Annealing` описує параметри промислового процесу відпалу сталі. Його можна завантажити з OpenML:

```
from sklearn.datasets import fetch_openml

data = fetch_openml(name='anneal', version=1, as_frame=True)
X, y = data.data, data.target
```

Основні характеристики:

- кількість спостережень: 898 (OpenML) або 798 (навчальна частина в UCI) — у звіті вказати, яку версію використано;
- 38 ознак: числові, бінарні та категоріальні;
- цільова змінна: багатокласова, із сильним дисбалансом — один клас домінує, а найменший містить лише кілька спостережень; один із класів, описаних у документації, у даних відсутній;
- багато ознак мають дуже високу частку пропусків, причому частина з них є структурними (див. документацію датасету `anneal.names`).

Ефект стелі

На цьому датасеті деревовидні моделі зазвичай досягають accuracy, близької до 1. Через це різниці між моделями невеликі й легко можуть бути шумом. Саме тому в роботі важливі macro F1, стандартне відхилення та статистична перевірка відмінностей, а не лише «у кого більше».

3 Хід роботи

3.1 Первинний аналіз даних

1. Завантажити датасет, визначити матрицю ознак X і цільову змінну y. Привести категоріальні ознаки до типу `object`.
2. Визначити кількість спостережень, ознак та їхні типи.
3. Обчислити частку пропусків для кожної ознаки. Ознаки, що є повністю порожніми або константними, видалити з обґрунтуванням.
4. Проаналізувати розподіл цільової змінної. Для класів із дуже малою кількістю спостережень обґрунтувати рішення (залишити, об'єднати або виключити) та пояснити, як воно вплине на крос-валідацію (див. «Типові помилки»).
5. Аналіз механізму пропусків. Для 3–5 ознак із найбільшою часткою пропусків побудувати таблицю спряженості «пропуск / не пропуск» $\times$ клас і перевірити незалежність критерієм χ^2 (або точним критерієм Фішера для малих частот). Зробити висновок: чи схожі пропуски на MCAR, чи вони пов'язані з класом?
6. Закодувати цільову змінну цілими числами $0, \ldots, K-1$ за допомогою `LabelEncoder` (цього вимагає XGBoost).

3.2 Поділ даних і пайплайн обробки

Виконати стратифікований поділ. Тестова вибірка використовується один раз — для фінальної оцінки в п. 3.6.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_enc, test_size=0.2, stratify=y_enc, random_state=SEED
)
```

Побудувати два варіанти обробки.

Варіант A (базова імпутація): числові ознаки — медіана; категоріальні — мода; one-hot кодування категоріальних ознак.

Варіант B (імпутація з індикаторами пропусків): як варіант A, плюс бінарні ознаки-індикатори для кожної ознаки, що містила пропуски у тренувальних даних.

```
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.impute import SimpleImputer, MissingIndicator
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

def build_preprocessor(with_indicators: bool) -> ColumnTransformer:
    transformers = [
        ('num', SimpleImputer(strategy='median'),
         make_column_selector(dtype_include='number')),
        ('cat', Pipeline([
            ('impute', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore',
                                     sparse_output=False)),
         ]), make_column_selector(dtype_exclude='number')),
    ]
    if with_indicators:
        transformers.append(
            ('miss', MissingIndicator(features='missing-only'),
             make_column_selector()))
    return ColumnTransformer(transformers)

def build_pipeline(name, with_indicators=True, params=None):
    return Pipeline([('prep', build_preprocessor(with_indicators)),
                     ('clf', make_model(name, params))])
```

Примітка. Починаючи з scikit-learn 1.3–1.4, `DecisionTreeClassifier` і `RandomForestClassifier` уміють працювати з `NaN` нативно, як і XGBoost. У цій роботі імпутація застосовується навмисно, щоб усі моделі отримували однакові вхідні дані і порівняння було контрольованим. Нативна обробка пропусків досліджується в додатковому завданні.

3.3 Базові моделі

Для кожного варіанту обробки навчити три моделі з типовими гіперпараметрами:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

def make_model(name, params=None):
    params = params or {}
    if name == 'DT':
        return DecisionTreeClassifier(random_state=SEED, **params)
    if name == 'RF':
```

```
        return RandomForestClassifier(random_state=SEED, n_jobs=-1,
                                      **params)
    return XGBClassifier(random_state=SEED, n_jobs=-1,
                         eval_metric='mlogloss', **params)
```

Оцінювання виконувати на тренувальній вибірці повторною стратифікованою крос-валідацією (5 блоків $\times$ 3 повторення) за метриками accuracy, macro F1 (основна) та balanced accuracy:

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_validate

scoring = {'acc': 'accuracy', 'f1m': 'f1_macro',
           'bacc': 'balanced_accuracy'}
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=SEED)
res = cross_validate(build_pipeline('RF', with_indicators=True),
                     X_train, y_train, cv=cv, scoring=scoring)
```

Усі моделі й варіанти мають оцінюватися на одному й тому самому об'єкті `cv` — це необхідно для парного статистичного порівняння в п. 3.5. Результати подати у форматі «середнє $\pm$ стандартне відхилення» (табл. 1 і 2).

3.4 Підбір гіперпараметрів за допомогою Optuna

Оптимізацію виконувати для варіанту B. Критично важливо, щоб у цільову функцію передавався весь пайплайн (обробка + модель), а не заздалегідь оброблені дані: інакше імпутер «бачить» валідаційні блоки і виникає витік даних.

```
import optuna
from sklearn.model_selection import StratifiedKFold, cross_val_score

def suggest_params(trial, name):
    if name == 'DT':
        return {
            'max_depth': trial.suggest_int('max_depth', 2, 20),
            'min_samples_split':
                trial.suggest_int('min_samples_split', 2, 20),
            'min_samples_leaf':
                trial.suggest_int('min_samples_leaf', 1, 10),
            'class_weight': trial.suggest_categorical(
                'class_weight', [None, 'balanced']),
        }
    if name == 'RF':
        return {
            'n_estimators':
                trial.suggest_int('n_estimators', 100, 500, step=50),
            'max_depth': trial.suggest_int('max_depth', 3, 30),
            'min_samples_leaf':
                trial.suggest_int('min_samples_leaf', 1, 10),
            'max_features': trial.suggest_categorical(
                'max_features', ['sqrt', 'log2', 0.5]),
            'class_weight': trial.suggest_categorical(
                'class_weight',
                [None, 'balanced', 'balanced_subsample']),
        }
    return {  # XGB
        'n_estimators': trial.suggest_int('n_estimators', 50, 400),
        'max_depth': trial.suggest_int('max_depth', 2, 10),
        'learning_rate':
            trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
```

```
        'colsample_bytree':
            trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'min_child_weight':
            trial.suggest_float('min_child_weight', 1, 10, log=True),
    }

def tune(name, X_tr, y_tr, n_trials=30):
    inner = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
    def objective(trial):
        pipe = build_pipeline(name, True, suggest_params(trial, name))
        return cross_val_score(pipe, X_tr, y_tr, cv=inner,
                               scoring='f1_macro').mean()
    study = optuna.create_study(
        direction='maximize',
        sampler=optuna.samplers.TPESampler(seed=SEED))
    study.optimize(objective, n_trials=n_trials)
    return study
```

Оцінка якості після оптимізації — вкладена крос-валідація. Для кожної моделі виконати зовнішню 5-блочну стратифіковану крос-валідацію; на кожному зовнішньому блоці запустити `tune()` лише на його тренувальній частині, а якість обчислювати на валідаційній:

```
from sklearn.metrics import f1_score

outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
nested_scores = []
for tr_idx, va_idx in outer.split(X_train, y_train):
    X_tr, y_tr = X_train.iloc[tr_idx], y_train[tr_idx]
    X_va, y_va = X_train.iloc[va_idx], y_train[va_idx]
    study = tune('XGB', X_tr, y_tr)
    model = build_pipeline('XGB', True, study.best_params).fit(X_tr, y_tr)
    nested_scores.append(f1_score(y_va, model.predict(X_va),
                                  average='macro'))
```

Результати вкладеної крос-валідації навести в табл. 3 разом зі значенням `study.best_value` однієї «звичайної» оптимізації на всій тренувальній вибірці. Порівняти ці дві величини та пояснити різницю.

Фінальні моделі. Запустити `tune()` на всій тренувальній вибірці, навчити кожну модель з найкращими параметрами (табл. 4) і зберегти. Для кожної моделі побудувати графіки `plot_optimization_history` та `plot_param_importances` з модуля `optuna.visualization.matplotlib` і коротко прокоментувати: чи збіглася оптимізація, які параметри найважливіші?

3.5 Статистичне порівняння

На основі результатів п. 3.3 (однакові блоки!) скоригованим t-тестом Наде–Бенжіо (розд. 1.3) перевірити при $\alpha = 0{,}05$:

1. чи значуща різниця macro F1 між варіантами A і B для кожної з трьох моделей;
2. чи значуща різниця між двома найкращими моделями у варіанті B.

Реалізувати тест самостійно (10–15 рядків коду, p-значення — через `scipy.stats.t`). Результати подати в табл. 5: $\bar{d}$, t, p, висновок.

3.6 Фінальна оцінка на тестовій вибірці

Для трьох фінальних моделей (варіант B, після оптимізації) обчислити accuracy, macro F1, balanced accuracy на тестовій вибірці (табл. 6), а також `classification_report` для найкращої моделі. Пояснити, чому тестові значення можуть відрізнятися від результатів крос-валідації (зокрема з огляду на кількість об'єктів найменших класів у тесті).

Для найкращої моделі побудувати абсолютну та нормалізовану (по рядках) матриці плутанини. Визначити, які класи найчастіше плутаються між собою.

3.7 Інтерпретація: важливість ознак

Для трьох фінальних моделей обчислити два види важливості ознак і побудувати графіки top-15:

1. MDI (`feature_importances_`) — важливість на основі зменшення неоднорідності; обчислюється на оброблених ознаках (після one-hot та з індикаторами); назви ознак — через `get_feature_names_out()`;
2. Permutation importance на тестовій вибірці (функція `permutation_importance` з модуля `sklearn.inspection`, `scoring='f1_macro'`, не менше 10 повторень), застосована до всього пайплайну — тобто на початкових ознаках.

Дати відповіді на запитання:

- чи потрапили індикатори пропусків (`missingindicator_*`) до top-15 за MDI? Що це означає з огляду на аналіз механізму пропусків (п. 3.1)?
- наскільки узгоджуються MDI та permutation importance? Чому MDI може бути зміщеною для ознак із великою кількістю унікальних значень [10]?
- чому важливості однієї ознаки можуть суттєво відрізнятися між DT, RF і XGB?

3.8 Аналіз розбіжностей між моделями

Об'єкт вважається *об'єктом розбіжності*, якщо прогнози трьох фінальних моделей не всі однакові (достатньо, щоб хоча б одна модель відрізнялася).

Знайти не менше трьох таких об'єктів у тестовій вибірці та подати їх у таблиці (табл. 7) за зразком:

Зразок таблиці розбіжностей

Об'єкт	Справжній клас	Decision Tree	Random Forest	XGBoost
#17	3	2	3	3
#48	5	5	2	5
#93	2	4	2	2

Якщо в тестовій вибірці розбіжностей менше трьох (через ефект стелі це цілком можливо), доповнити їх об'єктами з тренувальної вибірки, використовуючи out-of-fold прогнози (`cross_val_predict`) — і явно позначити це у звіті. Прогнози моделей на їхніх же тренувальних даних для цього не використовувати.

Для одного об'єкта розбіжності провести детальний аналіз:

1. вивести всі ознаки з початковими значеннями (до імпутації) і позначити, які з них були пропущені та якими значеннями їх заповнено;

2. за допомогою `shap.TreeExplainer` обчислити SHAP-значення [11, 12] для кожної моделі та навести п'ять ознак із найбільшим за модулем внеском у прогнозований клас (для багатокласових моделей масив SHAP-значень має форму `(n_samples, n_features, n_classes)`);

3. порівняти ці ознаки між моделями та пояснити причини розбіжності: роль імпутованих значень та індикаторів пропусків, «жорсткі» пороги одного дерева проти усереднення в ансамблях, близькість об'єкта до межі між класами (порівняти ймовірності `predict_proba`).

Типові помилки

1. **Витік під час імпутації:** `fit_transform` на всьому датасеті до поділу або до крос-валідації. Імпутація може бути лише всередині `Pipeline`.

2. **Витік під час оптимізації:** передача в Optuna заздалегідь оброблених `X_train_proc` або використання тестової вибірки в цільовій функції.

3. **Оптимістична оцінка:** подання `study.best_value` як оцінки якості моделі.

4. **Рідкісні класи та кількість блоків:** якщо клас має менше спостережень, ніж блоків крос-валідації, `StratifiedKFold` видає попередження, а XGBoost може завершитися помилкою, якщо клас відсутній у тренувальному блоці. Саме тому внутрішня крос-валідація має 3 блоки. Перевірте мінімальну кількість об'єктів класу на кожному рівні.

5. **Неоднакові блоки** для різних моделей унеможливлюють парне порівняння.

6. **Відсутність фіксованого** `SEED` — результати неможливо відтворити.

4 Вимоги до звіту

Звіт подається у форматі Jupyter Notebook, який має виконуватися без помилок від початку до кінця (`Restart & Run All`). До ноутбука додати `requirements.txt`. Кожен блок результатів супроводжується коротким текстовим коментарем (Markdown-клітинка).

Таблиці:

1. крос-валідація, варіант A, типові параметри;

2. крос-валідація, варіант B, типові параметри;

3. вкладена крос-валідація, варіант B, після оптимізації, + `best_value`;

4. найкращі гіперпараметри, знайдені Optuna;

5. результати статистичного порівняння;

6. якість фінальних моделей на тестовій вибірці;

7. об'єкти розбіжностей між прогнозами моделей.

Графіки:

1. розподіл класів цільової змінної;
2. частка пропусків за ознаками;
3. історія оптимізації та важливість гіперпараметрів (для кожної моделі);
4. важливості ознак: MDI та permutation importance (для кожної моделі);
5. абсолютна та нормалізована матриці плутанини найкращої моделі;
6. SHAP-внески для проаналізованого об'єкта розбіжності.

Висновок

У висновку (8–12 речень) відповісти на такі питання, спиряючись на власні числові результати:

1. Яка модель найкраща за macro F1 і наскільки стабільний цей результат? Чи є її перевага статистично значущою?
2. Як індикатори пропусків вплинули на якість? Чи узгоджується це з результатами аналізу механізму пропусків і з важливостями ознак?
3. Наскільки відрізняються `best_value` та оцінка вкладеної крос-валідації? Що це говорить про ризик оптимістичної оцінки?
4. Яка модель отримала найбільший приріст після оптимізації і чому?
5. Які класи найскладніші для класифікації і чому (кількість прикладів, схожість ознак)?
6. Що показав аналіз об'єктів розбіжностей?
7. Яку модель доцільно обрати для практичного використання з урахуванням якості, стабільності, інтерпретовності та обчислювальних витрат?

Примітка. Наперед визначеної «правильної» моделі не існує. Якщо результати відрізняються від очікуваних або різниці статистично незначущі — це не помилка, а результат, який треба проаналізувати.

Контрольні питання

1. Чим випадковий ліс відрізняється від окремого дерева рішень з погляду розкладу похибки на зміщення та дисперсію? Яку роль відіграє декореляція дерев?
2. У чому принципова різниця між бустингом і беггінгом? Який із методів схильніший до перенавчання з ростом кількості дерев і чому?
3. Чому macro F1 і balanced accuracy інформативніші за accuracy в умовах дисбалансу? У чому різниця між macro- та weighted-усередненням F1?
4. Опишіть механізми MCAR, MAR і MNAR. За якого механізму індикатори пропусків найкорисніші? Чи можна за даними однозначно відрізнити MAR від MNAR?
5. У яких місцях пайплайна може виникнути витік даних під час імпутації та оптимізації гіперпараметрів і як цьому запобігти?

6. Чому `best_value` оптимізації є оптимістично зміщеною оцінкою? Як працює вкладена крос-валідація і яку модель ми насправді оцінюємо в її зовнішньому циклі?

7. Чому звичайний парний t-тест не підходить для порівняння результатів крос-валідації? Яку поправку вносить тест Наде–Бенжіо?

8. Чим MDI-важливість відрізняється від permutation importance? Як корельовані ознаки впливають на кожну з них?

9. Що таке SHAP-значення і яку властивість (адитивність) вони задовольняють? Чим локальне пояснення відрізняється від глобальної важливості?

10. Як алгоритм XGBoost обробляє пропущені значення нативно (sparsity-aware split finding)?

11. Які переваги має Optuna (TPE-семплер, прунінг) порівняно з `GridSearchCV` та `RandomizedSearchCV`?

Література

[1] Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd ed. — New York: Springer, 2009. — 745 p. DOI: 10.1007/978-0-387-84858-7.

[2] Breiman L. Random Forests // Machine Learning. — 2001. — Vol. 45, No. 1. — P. 5–32. DOI: 10.1023/A:1010933404324.

[3] Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16). — New York: ACM, 2016. — P. 785–794. DOI: 10.1145/2939672.2939785.

[4] Pedregosa F., Varoquaux G., Gramfort A., Michel V., Thirion B., Grisel O., et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.

[5] Rubin D. B. Inference and Missing Data // Biometrika. — 1976. — Vol. 63, No. 3. — P. 581–592. DOI: 10.1093/biomet/63.3.581.

[6] Akiba T., Sano S., Yanase T., Ohta T., Koyama M. Optuna: A Next-generation Hyperparameter Optimization Framework // Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19). — 2019. — P. 2623–2631. DOI: 10.1145/3292500.3330701.

[7] Josse J., Prost N., Scornet E., Varoquaux G. On the consistency of supervised learning with missing values // arXiv preprint arXiv:1902.06931. — 2019.

[8] Cawley G. C., Talbot N. L. C. On over-fitting in model selection and subsequent selection bias in performance evaluation // Journal of Machine Learning Research. — 2010. — Vol. 11. — P. 2079–2107.

[9] Nadeau C., Bengio Y. Inference for the Generalization Error // Machine Learning. — 2003. — Vol. 52, No. 3. — P. 239–281. DOI: 10.1023/A:1024068626366.

[10] Strobl C., Boulesteix A.-L., Zeileis A., Hothorn T. Bias in random forest variable importance measures: Illustrations, sources and a solution // BMC Bioinformatics. — 2007. — Vol. 8. — Article 25. DOI: 10.1186/1471-2105-8-25.

[11] Lundberg S. M., Lee S.-I. A Unified Approach to Interpreting Model Predictions // Advances in Neural Information Processing Systems (NeurIPS 2017). — 2017. — Vol. 30. — P. 4765–4774. arXiv:1705.07874.

[12] Lundberg S. M., Erion G., Chen H., DeRose A., Turkel B., Lee S.-I. From local explanations to global understanding with explainable AI for trees // Nature Machine Intelligence. — 2020. — Vol. 2, No. 1. — P. 56–67. DOI: 10.1038/s42256-019-0138-9.